



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА №5

«Основы наложения текстур в OpenGL»

по дисциплине: *«Компьютерная графика»*

Выполнил: студент группы ИУК4-43Б

(Подпись)

Афанасьева Ю.Р.

(И.О. Фамилия)

Проверил:

(Подпись)

Тихонович Е.О.

(И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Цель: формирование практических навыков по работе с текстурами средствами OpenGL, их наложению на освещенные объекты подверженные проекционному сокращению.

Задачи: понимать принципы наложения растровых изображений на геометрические объекты, уметь реализовывать наложение текстур с использованием возможностей OpenGL, научиться использовать наложение множественных текстур, уметь создавать фотореалистичные сцены (корректное освещение), на которых присутствуют текстурированные геометрические объекты.

Вариант 3

- 1) Для [Листинга 1](#) на заданный в варианте геометрический объект наложить произвольную текстуру из файла в формате tga.
- 2) Используя [Листинг 2](#), для геометрических объектов реализовать вращение и показать действие одномерной текстуры. Цвет меняется плавно от темного до светлого оттенка. Количество градаций цвета не менее 12.
- 3) Используя [Листинг 3](#) создать коридор по форме указанной в варианте. Для каждой грани использовать свою собственную текстуру т.е. отдельный файл в формате tga..

Этапы выполнения работы:

1. На основе Листинга 1 была реализована загрузка растрового изображения из файла stone.tga и его передача в OpenGL с помощью функции glTexImage2D. Геометрия исходного примера была заменена кубом. Для каждой грани куба заданы нормаль и текстурные координаты, обеспечивающие корректное наложение каменной текстуры. Добавлено управление поворотом куба с помощью клавиш со стрелками.

2. Был построен тетраэдр и создана одномерная текстура из 16 градаций жёлтого цвета от тёмного оттенка к светлому. Для каждой грани вычисляется нормаль и определяется интенсивность освещения, которая используется в качестве одномерной текстурной координаты. С помощью функции таймера реализовано непрерывное вращение тетраэдра, позволяющее наблюдать изменение оттенков его граней.

3. Был создан коридор с пятиугольным сечением согласно варианту 3. Для пяти поверхностей коридора загружены отдельные текстуры из файлов brick.tga, wood.tga, stone.tga, tiles.tga и metal.tga. Текстурные объекты создаются и переключаются средствами OpenGL, а повторение изображений вдоль поверхностей формирует протяжённый коридор. Реализованы перемещение по коридору и поворот камеры с помощью клавиш со стрелками.

Задание 1: Для Листинга 1 на куб наложить произвольную текстуру из файла в формате tga.

Листинг 1

```
#include "glut.h"
#include <GL/glext.h>
#include <GL/glu.h>

#include <math.h>
#include <stdio.h>
#include <stdlib.h>

// Rotation amounts
static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

// Define targa header.
#pragma pack(1)
typedef struct
{
    GLbyte identsize;           // Size of ID field that follows header (0)
    GLbyte colorMapType;       // 0 = None, 1 = paletted
    GLbyte imageType;          // 0 = none, 1 = indexed, 2 = rgb, 3 = grey,
+8=rle
    unsigned short colorMapStart; // First colour map entry
    unsigned short colorMapLength; // Number of colors
    unsigned char colorMapBits;   // bits per palette entry
    unsigned short xstart;        // image x origin
    unsigned short ystart;        // image y origin
    unsigned short width;         // width in pixels
    unsigned short height;        // height in pixels
    GLbyte bits;                 // bits per pixel (8 16, 24, 32)
    GLbyte descriptor;           // image descriptor
} TGAHEADER;
#pragma pack(8)

////////////////////////////////////////
////////////////////////////////////////
// Allocate memory and load targa bits. Returns pointer to new buffer,
// height, and width of texture, and the OpenGL format of data.
// Call free() on buffer when finished!
// This only works on pretty vanilla targas... 8, 24, or 32 bit color
// only, no palettes, no RLE encoding.
GLbyte *gltLoadTGA(
    const char *szFileName,
    GLint *iWidth,
    GLint *iHeight,
    GLint *iComponents,
    GLenum *eFormat)
{
    FILE *pFile;
```

```

TGAHEADER tgaHeader;
unsigned long lImageSize;
short sDepth;
GLbyte *pBits = NULL;

// Default/Failed values
*iWidth = 0;
*iHeight = 0;
*eFormat = GL_BGR_EXT;
*iComponents = GL_RGB8;

// Attempt to open the file
pFile = fopen(szFileName, "rb");
if (pFile == NULL)
{
    return NULL;
}

// Read in header (binary)
fread(&tgaHeader, 18, 1, pFile);

// Do byte swap for big vs little endian
#ifdef __APPLE__
    BYTE_SWAP(tgaHeader.colorMapStart);
    BYTE_SWAP(tgaHeader.colorMapLength);
    BYTE_SWAP(tgaHeader.xstart);
    BYTE_SWAP(tgaHeader.ystart);
    BYTE_SWAP(tgaHeader.width);
    BYTE_SWAP(tgaHeader.height);
#endif

// Get width, height, and depth of texture
*iWidth = tgaHeader.width;
*iHeight = tgaHeader.height;
sDepth = tgaHeader.bits / 8;

// Put some validity checks here. Very simply, I only understand
// or care about 8, 24, or 32 bit targa's.
if (tgaHeader.bits != 8 && tgaHeader.bits != 24 && tgaHeader.bits != 32)
{
    fclose(pFile);
    return NULL;
}

// Calculate size of image buffer
lImageSize = tgaHeader.width * tgaHeader.height * sDepth;

// Allocate memory and check for success
pBits = (GLbyte *)malloc(lImageSize * sizeof(GLbyte));

if (pBits == NULL)

```

```

{
    fclose(pFile);
    return NULL;
}

// Read in the bits
// Check for read error. This should catch RLE or other
// weird formats that I don't want to recognize
if (fread(pBits, lImageSize, 1, pFile) != 1)
{
    free(pBits);
    fclose(pFile);
    return NULL;
}

// Set OpenGL format expected
switch (sDepth)
{
    case 3:
        *eFormat = GL_BGR_EXT;
        *iComponents = GL_RGB8;
        break;
    case 4:
        *eFormat = GL_BGRA_EXT;
        *iComponents = GL_RGBA8;
        break;
    case 1:
        *eFormat = GL_LUMINANCE;
        *iComponents = GL_LUMINANCE8;
        break;
}

// Done with File
fclose(pFile);

// Return pointer to image data
return pBits;
}

// Change viewing volume and viewport. Called when window is resized
void ChangeSize(int w, int h)
{
    // Prevent a divide by zero
    if (h == 0)
    {
        h = 1;
    }

    // Set Viewport to window dimensions
    glViewport(0, 0, w, h);
}

```

```

GLfloat fAspect = (GLfloat)w / (GLfloat)h;

// Reset coordinate system
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
// Produce the perspective projection
gluPerspective(35.0f, fAspect, 1.0, 40.0);

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
}

// This function does any needed initialization on the rendering
// context. Here it sets up and initializes the lighting for
// the scene.
void SetupRC()
{
    GLubyte *pBytes;
    GLint iWidth, iHeight, iComponents;
    GLenum eFormat;

    // Light values and coordinates
    GLfloat whiteLight[] = {0.5f, 0.5f, 0.5f, 1.0f};
    GLfloat sourceLight[] = {0.75f, 0.75f, 0.75f, 1.0f};
    GLfloat lightPos[] = {-10.0f, 5.0f, 5.0f, 1.0f};

    glEnable(GL_DEPTH_TEST); // Hidden surface removal
    glFrontFace(GL_CCW);     // Counter clockwise polygons face out
    glEnable(GL_CULL_FACE);  // Do not calculate inside of jet

    // Enable lighting
    glEnable(GL_LIGHTING);

    // Setup and enable light 0
    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, whiteLight);
    glLightfv(GL_LIGHT0, GL_AMBIENT, sourceLight);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, sourceLight);
    glLightfv(GL_LIGHT0, GL_POSITION, lightPos);
    glEnable(GL_LIGHT0);

    // Enable color tracking
    glEnable(GL_COLOR_MATERIAL);

    // Set Material properties to follow glColor values
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);

    // Black blue background
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);

    // Load texture
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

```

```

pBytes = (GLubyte *)glLoadTGA(
    "stone.tga", &iWidth, &iHeight, &iComponents, &eFormat);
glTexImage2D(
    GL_TEXTURE_2D,
    0,
    iComponents,
    iWidth,
    iHeight,
    0,
    eFormat,
    GL_UNSIGNED_BYTE,
    pBytes);
free(pBytes);

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
glEnable(GL_TEXTURE_2D);
}

// Respond to arrow keys
void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
    {
        xRot -= 5.0f;
    }

    if (key == GLUT_KEY_DOWN)
    {
        xRot += 5.0f;
    }

    if (key == GLUT_KEY_LEFT)
    {
        yRot -= 5.0f;
    }

    if (key == GLUT_KEY_RIGHT)
    {
        yRot += 5.0f;
    }

    xRot = (GLfloat)((const int)xRot % 360);
    yRot = (GLfloat)((const int)yRot % 360);
    glutPostRedisplay();
}

void RenderScene(void)

```

```

{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glPushMatrix();
    glTranslatef(0.0f, 0.0f, -5.0f);
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    glColor3f(1.0f, 1.0f, 1.0f);
    glBegin(GL_QUADS);

    // Передняя грань
    glNormal3f(0.0f, 0.0f, 1.0f);
    glTexCoord2f(0.0f, 0.0f);
    glVertex3f(-1.0f, -1.0f, 1.0f);
    glTexCoord2f(1.0f, 0.0f);
    glVertex3f(1.0f, -1.0f, 1.0f);
    glTexCoord2f(1.0f, 1.0f);
    glVertex3f(1.0f, 1.0f, 1.0f);
    glTexCoord2f(0.0f, 1.0f);
    glVertex3f(-1.0f, 1.0f, 1.0f);

    // Задняя грань
    glNormal3f(0.0f, 0.0f, -1.0f);
    glTexCoord2f(0.0f, 0.0f);
    glVertex3f(1.0f, -1.0f, -1.0f);
    glTexCoord2f(1.0f, 0.0f);
    glVertex3f(-1.0f, -1.0f, -1.0f);
    glTexCoord2f(1.0f, 1.0f);
    glVertex3f(-1.0f, 1.0f, -1.0f);
    glTexCoord2f(0.0f, 1.0f);
    glVertex3f(1.0f, 1.0f, -1.0f);

    // Левая грань
    glNormal3f(-1.0f, 0.0f, 0.0f);
    glTexCoord2f(0.0f, 0.0f);
    glVertex3f(-1.0f, -1.0f, -1.0f);
    glTexCoord2f(1.0f, 0.0f);
    glVertex3f(-1.0f, -1.0f, 1.0f);
    glTexCoord2f(1.0f, 1.0f);
    glVertex3f(-1.0f, 1.0f, 1.0f);
    glTexCoord2f(0.0f, 1.0f);
    glVertex3f(-1.0f, 1.0f, -1.0f);

    // Правая грань
    glNormal3f(1.0f, 0.0f, 0.0f);
    glTexCoord2f(0.0f, 0.0f);
    glVertex3f(1.0f, -1.0f, 1.0f);
    glTexCoord2f(1.0f, 0.0f);
    glVertex3f(1.0f, -1.0f, -1.0f);
    glTexCoord2f(1.0f, 1.0f);

```



```

glVertex3f(1.0f, 1.0f, -1.0f);
glTexCoord2f(0.0f, 1.0f);
glVertex3f(1.0f, 1.0f, 1.0f);

// Верхняя грань
glNormal3f(0.0f, 1.0f, 0.0f);
glTexCoord2f(0.0f, 0.0f);
glVertex3f(-1.0f, 1.0f, 1.0f);
glTexCoord2f(1.0f, 0.0f);
glVertex3f(1.0f, 1.0f, 1.0f);
glTexCoord2f(1.0f, 1.0f);
glVertex3f(1.0f, 1.0f, -1.0f);
glTexCoord2f(0.0f, 1.0f);
glVertex3f(-1.0f, 1.0f, -1.0f);

// Нижняя грань
glNormal3f(0.0f, -1.0f, 0.0f);
glTexCoord2f(0.0f, 0.0f);
glVertex3f(-1.0f, -1.0f, -1.0f);
glTexCoord2f(1.0f, 0.0f);
glVertex3f(1.0f, -1.0f, -1.0f);
glTexCoord2f(1.0f, 1.0f);
glVertex3f(1.0f, -1.0f, 1.0f);
glTexCoord2f(0.0f, 1.0f);
glVertex3f(-1.0f, -1.0f, 1.0f);

glEnd();
glPopMatrix();
glutSwapBuffers();
}

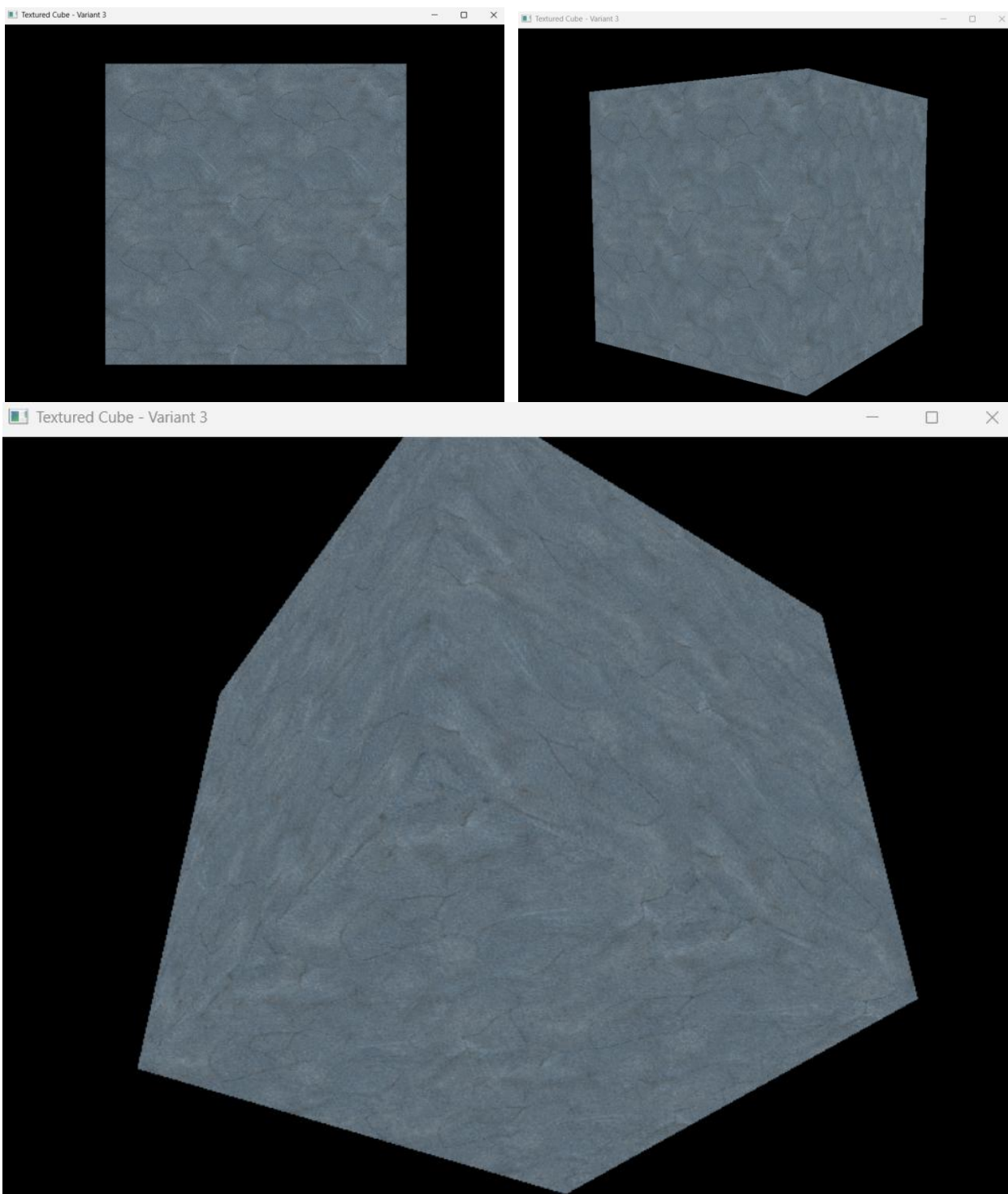
int main(int argc, char *argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Textured Cube - Variant 3");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);

    SetupRC();
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 1



Задание 2: Используя Листинг 2, для геометрических объектов реализовать вращение и показать действие одномерной текстуры. Цвет меняется плавно от темного до светлого оттенка. Количество градаций цвета не менее 12.

3) Желтый цвет, тетраэдр

Листинг 2

```
#include "glut.h"
#include <GL/glext.h>
#include <GL/glu.h>

#include <math.h>

static GLfloat rotationAngle = 0.0f;

typedef GLfloat Vector3[3];

struct Face
{
    GLint first;
    GLint second;
    GLint third;
};

static const Vector3 vertices[4] = {
    {1.0f, 1.0f, 1.0f},
    {-1.0f, -1.0f, 1.0f},
    {-1.0f, 1.0f, -1.0f},
    {1.0f, -1.0f, -1.0f}
};

static const Face faces[4] = {
    {0, 2, 1},
    {0, 1, 3},
    {0, 3, 2},
    {1, 2, 3}
};

void Normalize(Vector3 vector)
{
    GLfloat length = (GLfloat)sqrt(
        vector[0] * vector[0] +
        vector[1] * vector[1] +
        vector[2] * vector[2]);

    if (length == 0.0f)
    {
        return;
    }
}
```

```

        vector[0] /= length;
        vector[1] /= length;
        vector[2] /= length;
    }

void CalculateNormal(const Face &face, Vector3 normal)
{
    Vector3 firstEdge = {
        vertices[face.second][0] - vertices[face.first][0],
        vertices[face.second][1] - vertices[face.first][1],
        vertices[face.second][2] - vertices[face.first][2]
    };
    Vector3 secondEdge = {
        vertices[face.third][0] - vertices[face.first][0],
        vertices[face.third][1] - vertices[face.first][1],
        vertices[face.third][2] - vertices[face.first][2]
    };

    normal[0] = firstEdge[1] * secondEdge[2] - firstEdge[2] * secondEdge[1];
    normal[1] = firstEdge[2] * secondEdge[0] - firstEdge[0] * secondEdge[2];
    normal[2] = firstEdge[0] * secondEdge[1] - firstEdge[1] * secondEdge[0];
    Normalize(normal);
}

// Преобразование нормали в том же порядке, что и тетраэдра
void TransformNormal(const Vector3 normal, Vector3 transformedNormal)
{
    const GLfloat angleY = rotationAngle * 3.14159265f / 180.0f;
    const GLfloat angleX = -20.0f * 3.14159265f / 180.0f;

    Vector3 rotatedY = {
        normal[0] * (GLfloat)cos(angleY) + normal[2] * (GLfloat)sin(angleY),
        normal[1],
        -normal[0] * (GLfloat)sin(angleY) + normal[2] * (GLfloat)cos(angleY)
    };

    transformedNormal[0] = rotatedY[0];
    transformedNormal[1] =
        rotatedY[1] * (GLfloat)cos(angleX) - rotatedY[2] * (GLfloat)sin(angleX);
    transformedNormal[2] =
        rotatedY[1] * (GLfloat)sin(angleX) + rotatedY[2] * (GLfloat)cos(angleX);
}

GLfloat CalculateTextureCoordinate(const Vector3 normal)
{
    Vector3 lightDirection = {-0.4f, 0.7f, 1.0f};
    Vector3 transformedNormal;

    Normalize(lightDirection);
    TransformNormal(normal, transformedNormal);
}

```

```

GLfloat intensity =
    transformedNormal[0] * lightDirection[0] +
    transformedNormal[1] * lightDirection[1] +
    transformedNormal[2] * lightDirection[2];

if (intensity < 0.0f)
{
    intensity = 0.0f;
}

// Небольшая фоновая составляющая сохраняет видимость неосвещённых граней
return 0.15f + intensity * 0.85f;
}

void SetupRC()
{
    const GLint gradientCount = 16;
    GLubyte yellowGradient[gradientCount][3];

    // Создание не менее 12 оттенков жёлтого цвета от тёмного к светлому
    for (GLint i = 0; i < gradientCount; ++i)
    {
        GLfloat intensity = (GLfloat)i / (GLfloat)(gradientCount - 1);
        yellowGradient[i][0] = (GLubyte)(255.0f * intensity);
        yellowGradient[i][1] = (GLubyte)(220.0f * intensity);
        yellowGradient[i][2] = 0;
    }

    glClearColor(0.05f, 0.06f, 0.09f, 1.0f);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_CULL_FACE);
    glFrontFace(GL_CCW);

    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
    glTexImage1D(
        GL_TEXTURE_1D,
        0,
        GL_RGB,
        gradientCount,
        0,
        GL_RGB,
        GL_UNSIGNED_BYTE,
        yellowGradient);
    glTexParameteri(GL_TEXTURE_1D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_1D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_1D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
    glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_REPLACE);
    glEnable(GL_TEXTURE_1D);
}

void ChangeSize(int width, int height)

```

```

{
    if (height == 0)
    {
        height = 1;
    }

    glViewport(0, 0, width, height);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(40.0, (GLfloat)width / (GLfloat)height, 1.0, 20.0);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void RenderScene()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glPushMatrix();
    glTranslatef(0.0f, 0.0f, -6.0f);
    glRotatef(-20.0f, 1.0f, 0.0f, 0.0f);
    glRotatef(rotationAngle, 0.0f, 1.0f, 0.0f);

    glBegin(GL_TRIANGLES);
    for (GLint i = 0; i < 4; ++i)
    {
        Vector3 normal;
        CalculateNormal(faces[i], normal);

        // Одна координата выбирает оттенок из одномерной текстуры
        glNormal3fv(normal);
        glTexCoord1f(CalculateTextureCoordinate(normal));
        glVertex3fv(vertices[faces[i].first]);
        glVertex3fv(vertices[faces[i].second]);
        glVertex3fv(vertices[faces[i].third]);
    }
    glEnd();

    glPopMatrix();
    glutSwapBuffers();
}

void TimerFunction(int value)
{
    rotationAngle += 0.7f;
    if (rotationAngle >= 360.0f)
    {
        rotationAngle -= 360.0f;
    }
}

```

```

        glutPostRedisplay();
        glutTimerFunc(16, TimerFunction, 1);
    }

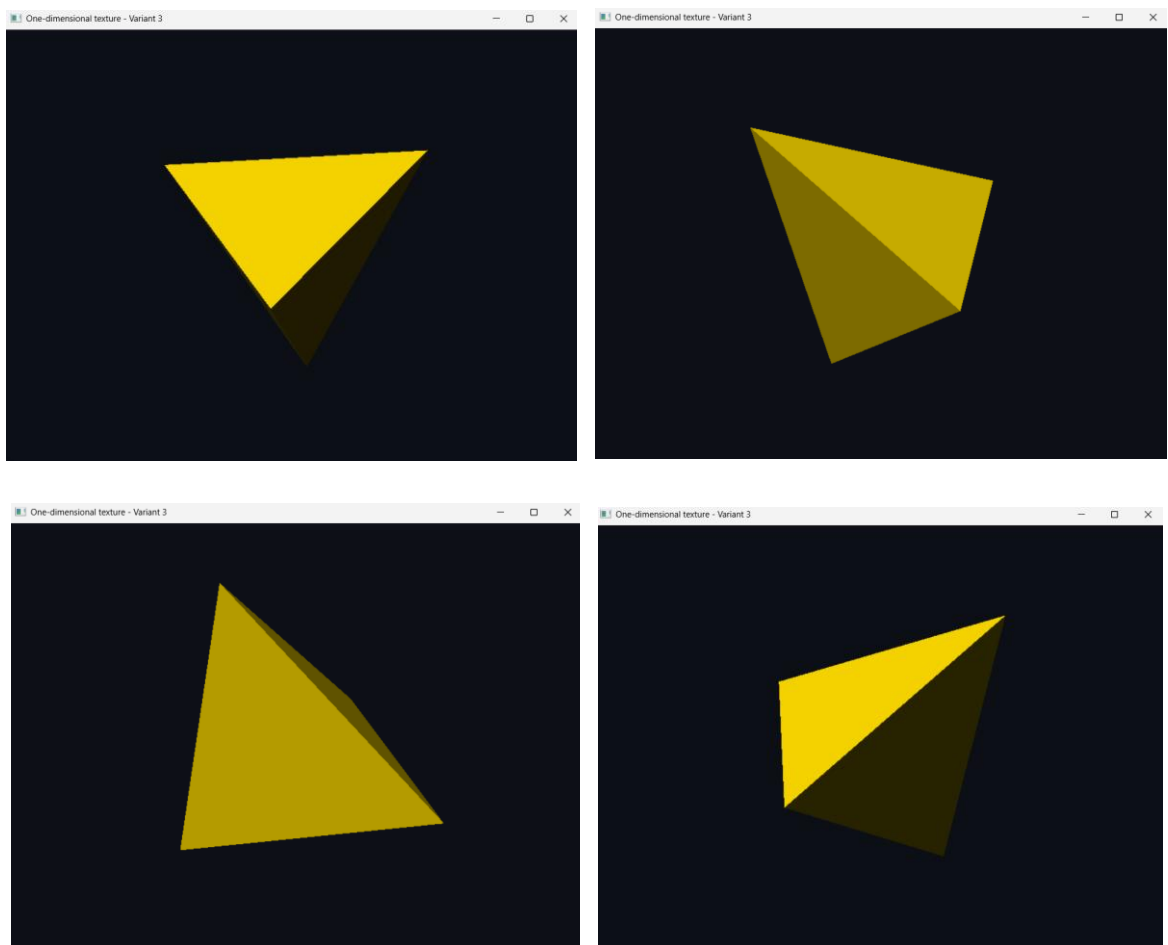
    int main(int argc, char *argv[])
    {
        glutInit(&argc, argv);
        glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
        glutInitWindowSize(800, 600);
        glutCreateWindow("One-dimensional texture - Variant 3");
        glutReshapeFunc(ChangeSize);
        glutDisplayFunc(RenderScene);

        SetupRC();
        glutTimerFunc(16, TimerFunction, 1);
        glutMainLoop();

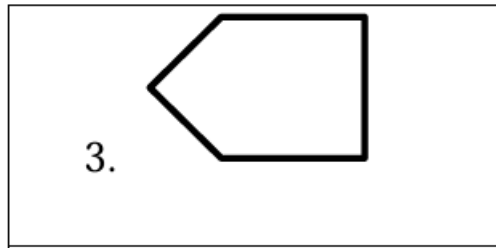
        return 0;
    }

```

Результаты работы программы 2



Задание 3: Используя Листинг 3 создать коридор по форме указанной в варианте. Для каждой грани использовать свою собственную текстуру т.е. отдельный файл в формате tga.



Листинг 3

```
#include "glut.h"
#include <GL/glext.h>
#include <GL/glu.h>

#include <stdio.h>

#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"

enum TextureIndex
{
    TEXTURE_BRICK,
    TEXTURE_WOOD,
    TEXTURE_STONE,
    TEXTURE_TILES,
    TEXTURE_METAL,
    TEXTURE_COUNT
};

static const char *textureFiles[TEXTURE_COUNT] = {
    "brick.tga",
    "wood.tga",
    "stone.tga",
    "tiles.tga",
    "metal.tga"
};

static GLuint textures[TEXTURE_COUNT];
static GLfloat corridorPosition = 0.0f;
static GLfloat viewRotation = 0.0f;

struct Point2D
{
    GLfloat x;
    GLfloat y;
};

// Пятиугольное сечение коридора для варианта 3
```

```

static const Point2D corridorPoints[5] = {
    {-3.0f, 0.0f},
    {-1.5f, 2.0f},
    {2.5f, 2.0f},
    {2.5f, -2.0f},
    {-1.5f, -2.0f}
};

bool LoadTexture(const char *fileName, GLuint texture)
{
    int width;
    int height;
    int channelCount;

    unsigned char *image = stbi_load(fileName, &width, &height, &channelCount,
3);
    if (image == NULL)
    {
        fprintf(stderr, "Не удалось загрузить текстуру %s\n", fileName);
        return false;
    }

    glBindTexture(GL_TEXTURE_2D, texture);
    glTexImage2D(
        GL_TEXTURE_2D,
        0,
        GL_RGB,
        width,
        height,
        0,
        GL_RGB,
        GL_UNSIGNED_BYTE,
        image);

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);

    stbi_image_free(image);
    return true;
}

void SetupRC()
{
    glClearColor(0.02f, 0.02f, 0.03f, 1.0f);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_TEXTURE_2D);
    glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_REPLACE);
    glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
}

```

```

    stbi_set_flip_vertically_on_load(1);
    glGenTextures(TEXTURE_COUNT, textures);

    for (GLint i = 0; i < TEXTURE_COUNT; ++i)
    {
        LoadTexture(textureFiles[i], textures[i]);
    }
}

void ShutdownRC()
{
    glDeleteTextures(TEXTURE_COUNT, textures);
}

void ChangeSize(int width, int height)
{
    if (height == 0)
    {
        height = 1;
    }

    glViewport(0, 0, width, height);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60.0, (GLfloat)width / (GLfloat)height, 0.5, 180.0);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void DrawSurface(
    const Point2D &first,
    const Point2D &second,
    GLfloat nearZ,
    GLfloat farZ,
    GLuint texture)
{
    glBindTexture(GL_TEXTURE_2D, texture);

    glBegin(GL_QUADS);
    glTexCoord2f(0.0f, 0.0f);
    glVertex3f(first.x, first.y, nearZ);
    glTexCoord2f(1.0f, 0.0f);
    glVertex3f(second.x, second.y, nearZ);
    glTexCoord2f(1.0f, 20.0f);
    glVertex3f(second.x, second.y, farZ);
    glTexCoord2f(0.0f, 20.0f);
    glVertex3f(first.x, first.y, farZ);
    glEnd();
}

```

```

void DrawCorridor(GLfloat nearZ, GLfloat farZ)
{
    // Для каждой поверхности используется отдельный текстурный объект
    for (GLint i = 0; i < 5; ++i)
    {
        DrawSurface(
            corridorPoints[i],
            corridorPoints[(i + 1) % 5],
            nearZ,
            farZ,
            textures[i]);
    }
}

void RenderScene()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glPushMatrix();
    glRotatef(viewRotation, 0.0f, 1.0f, 0.0f);
    glTranslatef(0.0f, 0.0f, corridorPosition);

    // Два последовательных участка скрывают переход при движении
    DrawCorridor(2.0f, -78.0f);
    DrawCorridor(-78.0f, -158.0f);

    glPopMatrix();
    glutSwapBuffers();
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
    {
        corridorPosition += 1.0f;
    }

    if (key == GLUT_KEY_DOWN)
    {
        corridorPosition -= 1.0f;
    }

    if (key == GLUT_KEY_LEFT)
    {
        viewRotation += 2.0f;
    }

    if (key == GLUT_KEY_RIGHT)
    {
        viewRotation -= 2.0f;
    }
}

```

```

    }

    if (corridorPosition >= 80.0f)
    {
        corridorPosition -= 80.0f;
    }
    else if (corridorPosition < 0.0f)
    {
        corridorPosition += 80.0f;
    }

    glutPostRedisplay();
}

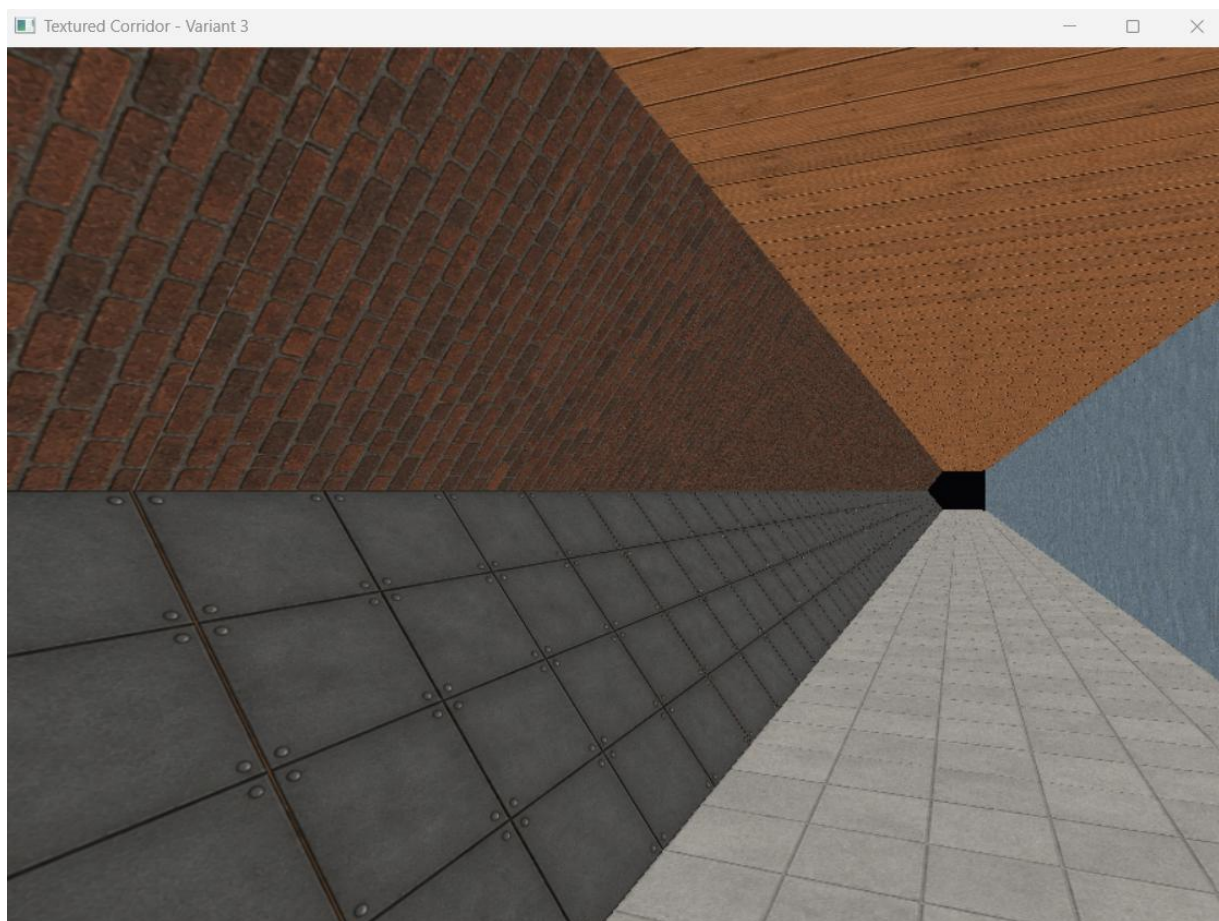
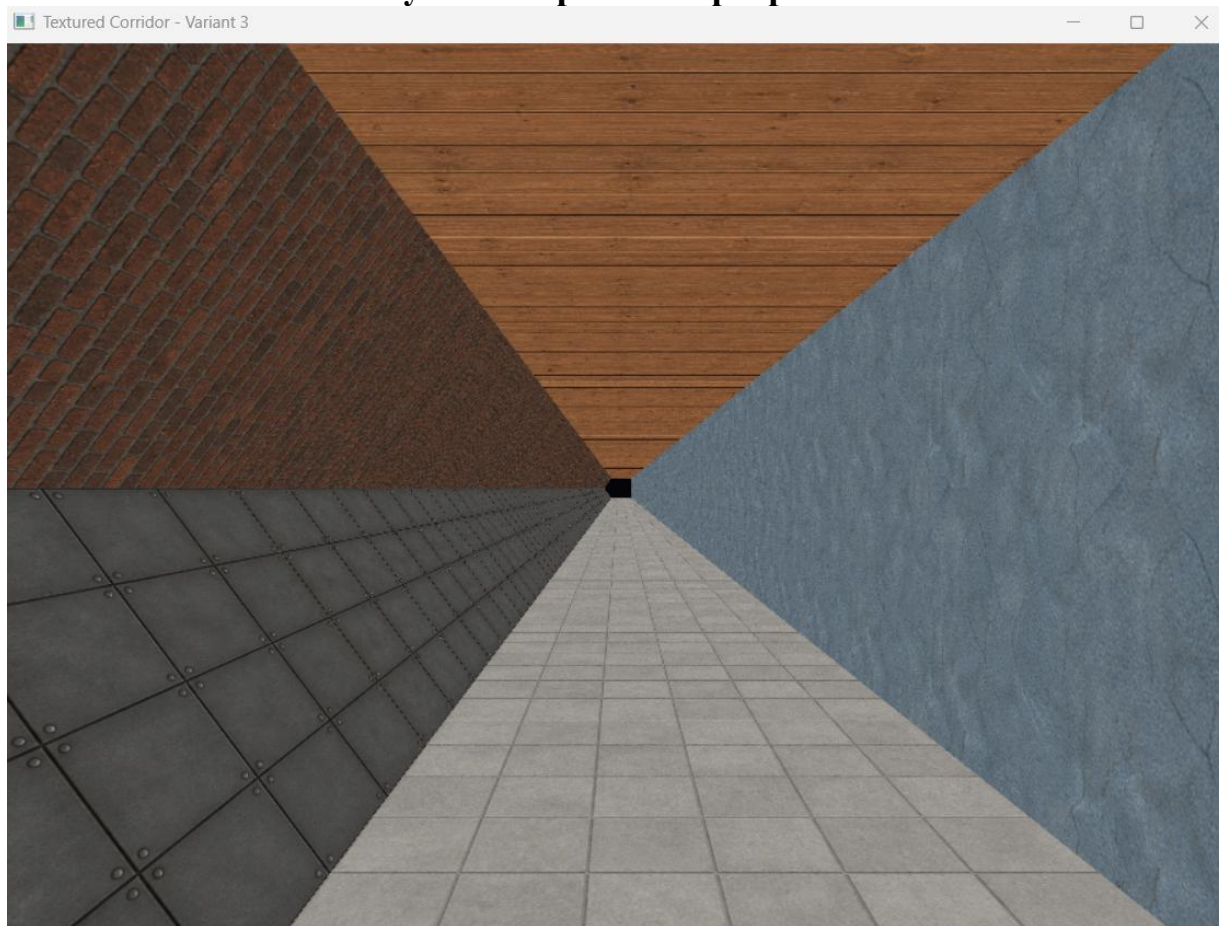
int main(int argc, char *argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(900, 650);
    glutCreateWindow("Textured Corridor - Variant 3");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);
    glutSpecialFunc(SpecialKeys);

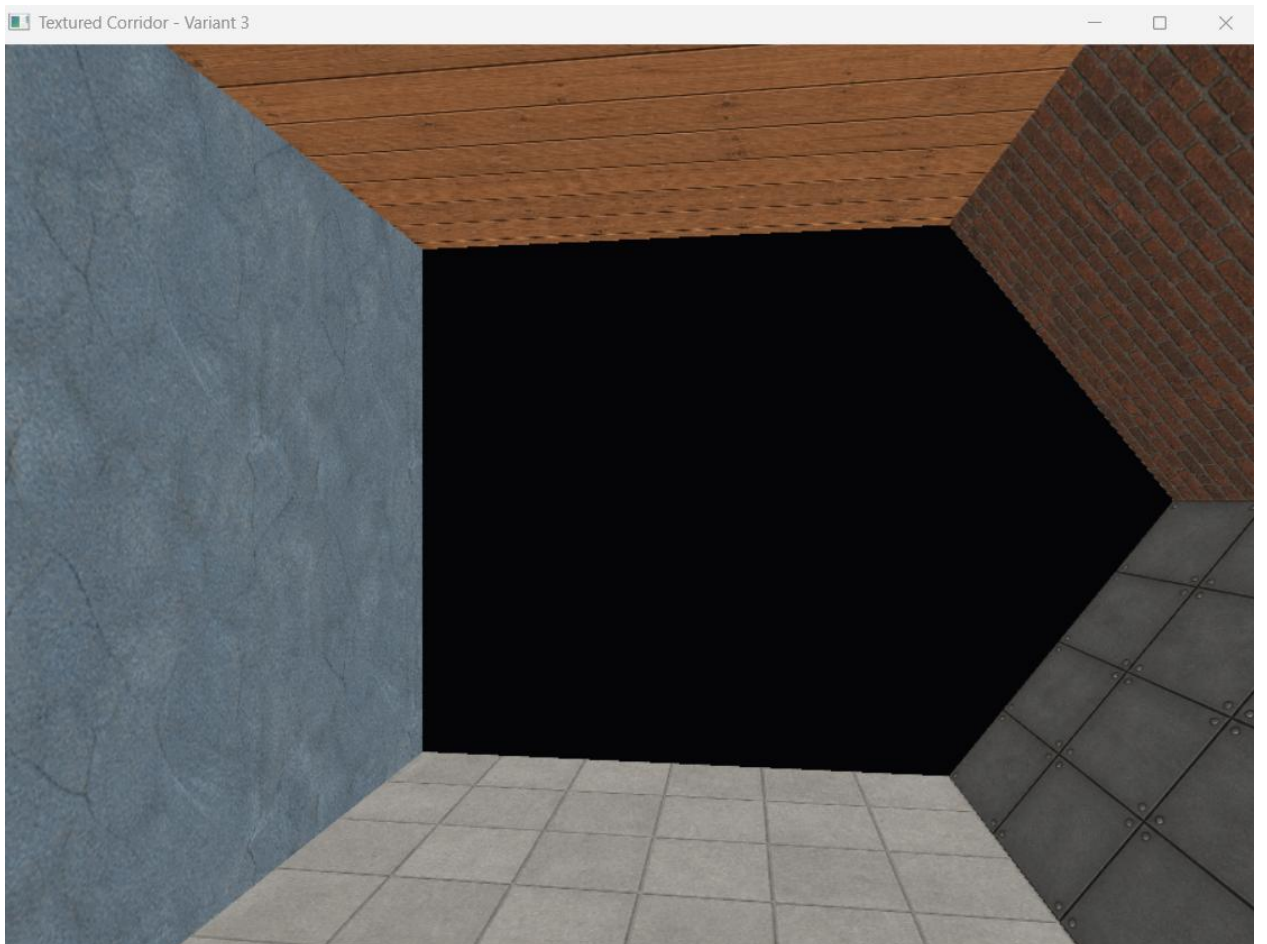
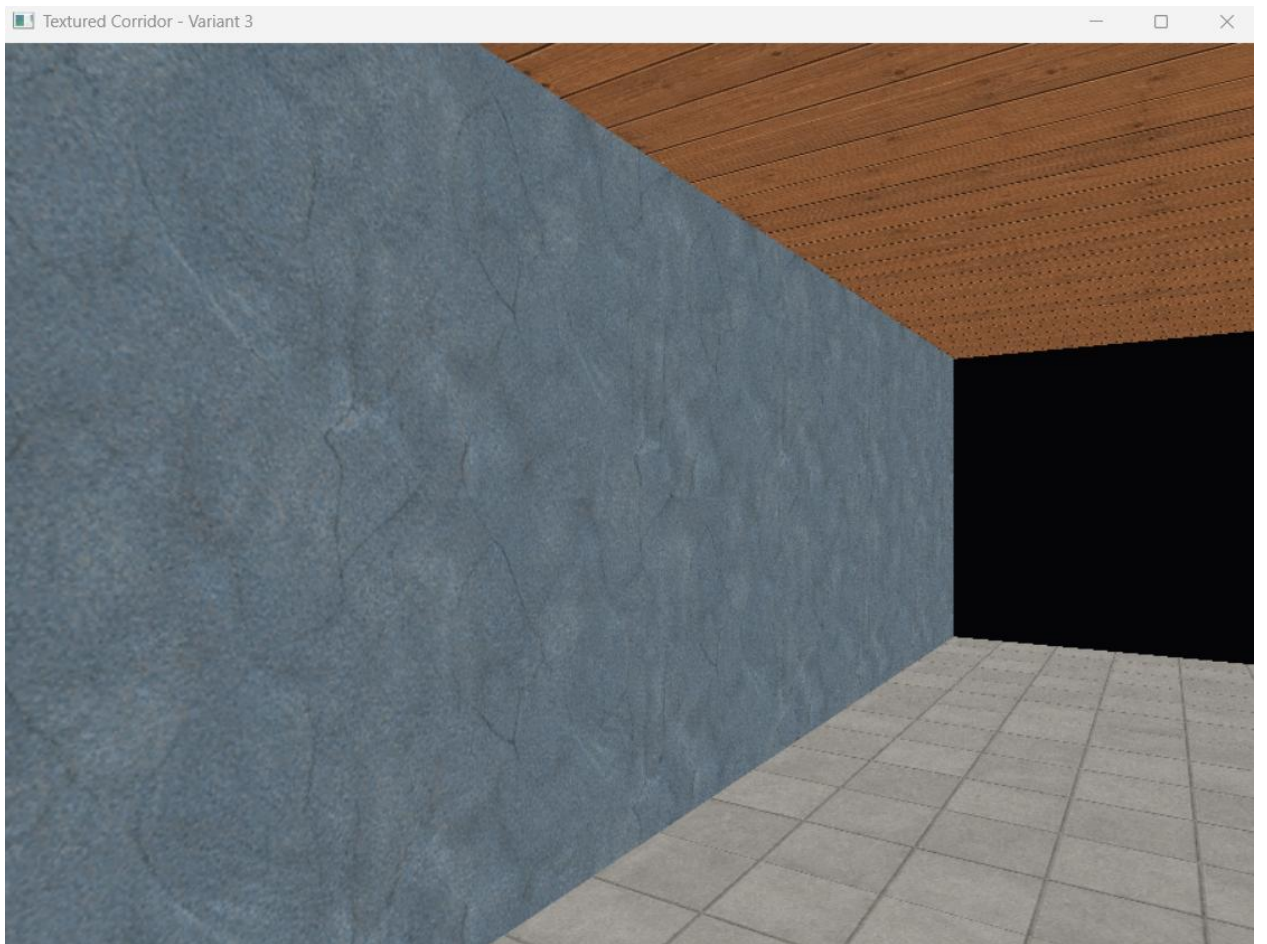
    SetupRC();
    glutMainLoop();
    ShutdownRC();

    return 0;
}

```

Результаты работы программы 3





Вывод: в ходе работы были получены практические навыки по работе с текстурами средствами OpenGL, их наложению на освещенные объекты подверженные проекционному сокращению.